

# Getränkebestellungen am Rattenfest

Studienarbeit  
Herbstsemester 2026



Version: Lokaler Build  
Datum: 01.01.1980

**Team:** Andrin Klarer  
Jasmin Fässler  
**Betreuer:** Frieder Loch

Departement Informatik  
Ostschweizer Fachhochschule



# Inhaltsverzeichnis

|       |                                                     |    |
|-------|-----------------------------------------------------|----|
| 1     | Problemstellung .....                               | 6  |
| 1.1   | Ausgangslage .....                                  | 6  |
| 1.1.1 | Systemkontext und Domäne .....                      | 6  |
| 1.2   | Ziel .....                                          | 6  |
| 1.2.1 | Akteure .....                                       | 6  |
| 1.2.2 | Risiken und Unsicherheiten .....                    | 7  |
| 1.2.3 | User Stories und Functional Requirements .....      | 7  |
| 1.2.4 | Non-Functional Requirements .....                   | 10 |
| 1.2.5 | Anforderungen (To be removed) .....                 | 12 |
| 1.2.6 | Functional Requirements .....                       | 13 |
| 1.2.7 | Non-Functional Requirements .....                   | 14 |
| 1.3   | Anwendungsszenarien (detaillierter) .....           | 14 |
| 1.4   | Vorhandene Lösungswege .....                        | 15 |
| 1.4.1 | Webshops .....                                      | 15 |
| 2     | Analyse .....                                       | 15 |
| 2.1   | Marktanalyse .....                                  | 16 |
| 2.1.1 | Sum-Up .....                                        | 16 |
| 2.2   | Recherche User Experience während Rattenfest .....  | 16 |
| 2.2.1 | Definition .....                                    | 16 |
| 2.2.2 | Erkenntnisse .....                                  | 16 |
| 2.2.3 | wichtige Probleme .....                             | 16 |
| 3     | Vorarbeiten .....                                   | 16 |
| 4     | Methode .....                                       | 16 |
| 4.1   | Design und Architektur .....                        | 16 |
| 4.1.1 | Prototyp / Skizze der Lösung .....                  | 16 |
| 4.1.2 | UI und Backend Konzept (Software-Architektur) ..... | 16 |
| 4.2   | Implementation .....                                | 16 |
| 5     | Ergebnisse .....                                    | 16 |
| 5.1   | Evaluation .....                                    | 16 |
| 5.1.1 | Functional Requirements .....                       | 16 |
| 5.1.2 | Non-Functional Requirements .....                   | 16 |
| 6     | Fazit und Ausblick .....                            | 16 |
| 6.1   | Retrospektive .....                                 | 16 |
| 6.2   | Zukünftige Weiterentwicklung .....                  | 16 |
| 7     | Projektinformationen .....                          | 17 |
| 7.1   | Projektplan .....                                   | 17 |
| 7.2   | Langfristiger Zeitplan .....                        | 17 |
| 7.3   | Kurzfristiger Plan .....                            | 18 |
| 7.3.1 | Kanban Board .....                                  | 19 |
| 8     | Hilfsmittelverzeichnis .....                        | 20 |
| 9     | Bilderverzeichnis .....                             | 20 |
| 10    | Tabellenverzeichnis .....                           | 20 |
| 11    | Bibliografie .....                                  | 21 |
| 12    | Anhang A .....                                      | 21 |
| 12.1  | APIs .....                                          | 21 |
| 12.2  | Screenshots .....                                   | 21 |
| 12.3  | Testreports .....                                   | 21 |

|      |                                  |    |
|------|----------------------------------|----|
| 13   | Anhang B .....                   | 21 |
| 13.1 | Meeting-Protokolle .....         | 21 |
| 13.2 | Eigenständigkeitserklärung ..... | 21 |
| 13.3 | Persönliche Bericht .....        | 21 |



# **Abstract**

## **Management Summary**

**Vorgehen und Technologien**

**Ergebnisse**

# 1 Problemstellung

## 1.1 Ausgangslage

Das Rattenfest ist ein jährlich stattfindendes Festival, das von Studierenden der OST organisiert wird und rund 3'000 Personen besucht. Die Getränke werden an mehreren Bars ausgeschenkt, die unabhängig vom Rattenfest betrieben werden. Damit nicht jede Bar ihre Versorgung selbst organisieren muss, betreibt das Rattenfest ein zentrales Getränkelager, aus dem die Bars während des Fests laufend beziehen.

Dieser Prozess wurde in den letzten Jahren versucht zunehmend zu digitalisieren, die bisherigen Lösungen blieben jedoch unzureichend.

### 1.1.1 Systemkontext und Domäne

Es gibt eine Website des Rattenfest [rattenfest.ch](http://rattenfest.ch), diese wird über Hospoint gehostet. Das Ticketsystem und die Bezahlung läuft extern über einen Anbieter. Die Kommunikation intern läuft über Microsoft Teams, E-Mails und Chatdienste wie Whatsapp. Dateien sind unter Microsoft Teams abgelegt, es gibt keinen Dateiserver. Es gibt bereits einen Prototyp für das Barsystem und als Hilfe für den OK im Vorverkauf wurde bereits eine kleine Website getestet. So sind die Schnittstellen vom Rattenfest sehr flexibel und die verschiedenen Services können unabhängig voneinander gewechselt werden.

## 1.2 Ziel

Ziel dieser Arbeit ist eine Webapplikation, die den gesamten Lebenszyklus der Getränke abbildet, von der ersten Bedarfsschätzung bis zur Abrechnung nach dem Fest:

- **Vor dem Fest** erfassen die Bars ihre Bedarfsschätzung in der Applikation. Das OK prüft und korrigiert diese und leitet daraus die Bestellung beim Getränkehändler ab. Die gelieferte Ware bildet den Anfangsbestand des Lagers.
- **Während des Fests** bestellen die Bars laufend aus diesem Bestand. Das Lagerteam stellt die Bestellungen bereit und informiert die Bars über den Status.
- **Nach dem Fest** lässt sich für jede Bar eine detaillierte Abrechnung erstellen.

Neben der Funktionalität soll die Lösung auf das Rattenfest zugeschnitten und für den langfristigen Einsatz geeignet sein. Dabei sind drei Eigenschaften besonders wichtig:

- **Bedienbarkeit** unter den Bedingungen am Fest: dunkel, laut, unter Zeitdruck und mit Personal, dessen Aufmerksamkeit im Verlauf des Abends nachlässt.
- **Zuverlässigkeit** während des Fests. Sobald die Bars auf den manuellen Weg ausweichen müssen, verliert die Applikation ihren Zweck.
- **Verwendbarkeit und Wartbarkeit** durch ein jährlich wechselndes OK mit unterschiedlichen technischen Kenntnissen.

### 1.2.1 Akteure

- **Bar-Team:** bestellt Getränke für seine Bar und holt sie ab.
- **Lagerteam:** nimmt Bestellungen entgegen, stellt sie bereit und übergibt sie.
- **Getränkechef (RF-OK):** verwaltet Festausgabe, Sortiment, Bars, Rücknahmen und Abrechnung.

## 1.2.2 Risiken und Unsicherheiten

Risiko, dass es auf einer Plattform/Betriebssystem/Bildschirmgröße nicht funktioniert wie gedacht  
Risiko, dass es nicht alle Personen einrichten können

Spezielle Umgebung: während Rattenfest

- ausserhalb von der Website: Tipps für Bars, wie sie das Gerät mit der Website einrichten können, damit es möglichst integriert und nicht lästig ist (Gerät, Halterung, Rolle zuweisen ?)
- Bedienung von Website (User Experience) eng, dunkel, laut, Bedienung eingeschränkt

## 1.2.3 User Stories und Functional Requirements

### 1.2.3.1 Vorbereitung

**US-01:** Als RF-OK möchte ich eine neue Festaussgabe anlegen, damit die Daten jedes Jahrgangs getrennt bleiben und vergleichbar werden.

- **FR-01:** Das System muss erlauben, eine Festaussgabe anzulegen und als aktiv zu setzen. Sortiment, Bestände, Bestellungen und Rückgaben sind jeweils einer Festaussgabe zugeordnet.
- **FR-02:** Das System muss erlauben, Sortiment und Bars einer früheren Festaussgabe als Ausgangslage zu übernehmen.

**US-02:** Als RF-OK möchte ich die Bars erfassen und ihnen Zugänge geben, damit Bestellungen eindeutig zugeordnet sind.

- **FR-03:** Das System muss erlauben, Bars mit Bezeichnung und Kontaktangabe zu erfassen, zu ändern und zu deaktivieren.
- **FR-04:** Das System muss pro Bar einen Zugang als teilbaren Link erzeugen und erlauben, diesen neu zu erzeugen, womit der bisherige ungültig wird.

**US-03:** Als RF-OK möchte ich das Getränkesortiment unseres Lieferanten den Bars zur Verfügung stellen.

- **FR-05:** Das System muss erlauben, Getränke mit Bezeichnung, Gebindegröße und Preis zu erfassen.
- **FR-06:** Das System muss erlauben, ein Sortiment aus einer Datei zu importieren.

**US-04:** Als Bar-Team möchte ich meinen Bedarf vor dem Fest schätzen, damit die benötigte Ware eingekauft wird.

- **FR-07:** Das System muss erlauben, pro Bar eine Bedarfsschätzung über das Sortiment zu erfassen und bis zu einem Stichtag zu ändern.

**US-05:** Als RF-OK möchte ich die Schätzungen prüfen und zusammenfassen, damit ich daraus die Bestellung beim Getränkehändler ableiten kann.

- **FR-08:** Das System muss die Schätzungen aller Bars darstellen und einzeln korrigierbar machen.

**US-06:** Als RF-OK möchte ich die gelieferte Ware einlagern, damit der Lagerbestand von Beginn an stimmt. **FR-09:** Das System muss den Anfangsbestand aus der Bestellung beim Händler übernehmen.

### 1.2.3.2 Bestellung während des Fests

**US-07:** Als Bar-Team möchte ich online Getränke bestellen.

- **FR-10:** Das System muss erlauben, eine Bestellung über mehrere Artikel mit Mengenangabe abzuschicken.
- **FR-11:** Das System darf nur Mengeneingaben erlauben die den Gebindegrößen entsprechen.
- **FR-12:** Das System muss verhindern, dass mehr bestellt wird, als im Lager verfügbar ist.
- **FR-13:** Das System muss einen Hinweis anzeigen, wenn eine Bestellung einen Artikel enthält, der sich von der Reservation der Bar unterscheidet.

**US-08:** Als Bar-Team möchte ich eine abgesendete Bestellung stornieren können.

- **FR-14:** Das System muss erlauben, eine Bestellung zu stornieren, solange das Lagerteam noch nicht bearbeitet hat.

**US-09:** Als Bar-Team möchte ich eine frühere Bestellung wiederholen.

- **FR-15:** Das System muss die Bestellhistorie der eigenen Bar anzeigen und erlauben, einen früheren Eintrag als Vorlage zu übernehmen und vor dem Absenden anzupassen.

### **1.2.3.3 Bearbeitung und Abholung**

**US-10:** Als Lagerteam möchte ich eingehende Bestellungen sofort sehen, damit ich ohne Verzögerung mit dem Bereitstellen beginnen kann.

- **FR-16:** Das System muss Bestellungen und deren Statusänderungen bei Bar und Lagerteam ohne manuelles Neuladen aktualisieren.
- **FR-17:** Das System muss offene Bestellungen in der Reihenfolge ihres Eingangs darstellen.

**US-11:** Als Bar-Team möchte ich wissen, wann meine Bestellung bereitsteht.

- **FR-18:** Das System muss erlauben, eine Bestellung als abholbereit zu markieren.
- **FR-19:** Das System muss erlauben, die Abholung zu bestätigen, womit die Bestellung abgeschlossen wird.
- **FR-20:** Das System muss eine Benachrichtigung an das Bar-Team senden, wenn die Bestellung abholbereit ist.

**US-12:** Als Lagerteam möchte ich Abweichungen festhalten, damit sie bei der Abrechnung nachvollziehbar sind.

- **FR-21:** Das System muss erlauben, eine Bestellung mit Begründung zu stornieren oder als nicht abgeholt zu kennzeichnen.

**US-13:** Als Bar-Team möchte ich bei technischen Problemen oder Zwischenfällen jemanden erreichen.

- **FR-22:** Das System muss eine Kontaktmöglichkeit zum Lagerteam und Sicherheitsverantwortlichen anzeigen.

**US-14:** Als Lagerteam möchte ich eine Pause einlegen, damit die Bars wissen, dass Bestellungen vorübergehend nicht bearbeitet werden.

- **FR-23:** Das System muss erlauben, das Lager als vorübergehend nicht besetzt zu markieren, was den Bars angezeigt wird.

### **1.2.3.4 Rücknahme und Abrechnung**

**US-15:** Als Bar-Team möchte ich nicht verkaufte Ware zurückgeben, damit sie mir nicht verrechnet wird.

- **FR-24:** Das System muss erlauben, eine Rückgabe analog zu einer Bestellung zu erfassen, begrenzt auf die von der Bar bezogenen Artikel und Mengen.
- **FR-25:** Das System muss verlangen, dass das RF-OK eine Rückgabe bestätigt, bevor sie dem Lagerbestand gutgeschrieben wird.

**US-16:** Als RF-OK möchte ich nach dem Fest einfach und schnell pro Bar abrechnen.

- **FR-26:** Das System muss pro Bar eine Aufstellung aller bezogenen und zurückgegebenen Artikel mit Betrag erzeugen.
- **FR-27:** Das System muss Abrechnung und Rohdaten der Bezüge als Datei exportieren.

**US-17:** Als RF-OK möchte ich den Verbrauch mehrerer Feste vergleichen, damit die Schätzung im Folgejahr besser wird.

- **FR-28:** Das System muss die Daten vergangener Feste erhalten und pro Artikel vergleichbar darstellen.

### 1.2.3.5 Klassifikation der Functional Requirements

Die Functional Requirements sind nach der MoSCoW-Methode in drei Kategorien eingeteilt.

| Kategorie | Bedeutung                                                                                                                  |
|-----------|----------------------------------------------------------------------------------------------------------------------------|
| Muss      | Ohne diese Anforderungen erfüllt das System seinen Zweck nicht.                                                            |
| Soll      | Wichtig für Bedienbarkeit, Effizienz oder Zuverlässigkeit. Das System funktioniert ohne sie, aber mit geringerer Qualität. |
| Kann      | Wünschenswerte Ergänzung, die entfällt, wenn die Zeit nicht reicht.                                                        |

Tabelle 1: Kategorien der MoSCoW-Klassifikation

| ID    | Kurzbezeichnung                         | Priorität |
|-------|-----------------------------------------|-----------|
| FR-01 | Festausgabe anlegen und aktiv setzen    | Muss      |
| FR-02 | Stammdaten aus Vorjahr übernehmen       | Soll      |
| FR-03 | Bars verwalten                          | Muss      |
| FR-04 | Bar-Zugang erzeugen und erneuern        | Muss      |
| FR-05 | Sortiment verwalten                     | Muss      |
| FR-06 | Sortiment importieren                   | Kann      |
| FR-07 | Bedarfsschätzung erfassen               | Muss      |
| FR-08 | Schätzungen aggregieren und korrigieren | Muss      |
| FR-09 | Anfangsbestand erfassen                 | Muss      |
| FR-10 | Bestellung absenden                     | Muss      |
| FR-11 | Schrittweite für Mengeneingabe          | Soll      |
| FR-12 | Bestand als Obergrenze                  | Muss      |
| FR-13 | Hinweis bei auffälliger Bestellung      | Kann      |
| FR-14 | Bestellung stornieren (Bar)             | Muss      |
| FR-15 | Historie anzeigen und wiederverwenden   | Soll      |
| FR-16 | Aktualisierung ohne Neuladen            | Muss      |
| FR-17 | Offene Bestellungen nach Eingang        | Muss      |
| FR-18 | Als abholbereit markieren               | Muss      |
| FR-19 | Abholung bestätigen                     | Muss      |
| FR-20 | Benachrichtigung ausserhalb der App     | Kann      |
| FR-21 | Bestellung kennzeichnen (Lager)         | Soll      |
| FR-22 | Kontaktmöglichkeit                      | Soll      |
| FR-23 | Lager als nicht besetzt markieren       | Kann      |
| FR-24 | Rückgabe erfassen                       | Soll      |
| FR-25 | Rückgabe bestätigen                     | Soll      |
| FR-26 | Abrechnung pro Bar                      | Muss      |
| FR-27 | Datenexport                             | Soll      |
| FR-28 | Vergleich über Festausgaben             | Kann      |

Tabelle 2: Klassifikation der Functional Requirements

## 1.2.4 Non-Functional Requirements

Die folgenden Anforderungen gelten für das System als Ganzes. Zu jeder ist angegeben, auf welche Functional Requirements sie sich bezieht und woran ihre Erfüllung gemessen wird. Da das Rattenfest nach Abgabe dieser Arbeit stattfindet, sind alle Anforderungen ohne das Fest überprüfbar. Im Vordergrund stehen die Zuverlässigkeit während des Fests und die Betreibbarkeit über mehrere Jahre. Skalierbarkeit, Redundanz und Barrierefreiheit sind bewusst niedrig angesetzt, da das System nicht öffentlich zugänglich ist und von weniger als 20 gleichzeitig verbundenen Personen genutzt wird.

### **NFR-01 (Konsistenz der Bestandsführung)** betrifft FR-10, FR-12, FR-24

Eine Bestellung wird vollständig oder gar nicht verbucht, und auch bei wiederholtem Absenden genau einmal. Bei gleichzeitigen Bestellungen mehrerer Bars auf denselben Artikel darf die Summe der bestätigten Bezüge den verfügbaren Bestand nicht überschreiten und der Bestand nicht negativ werden.

*Messung:* Manueller Test, der zwei Bestellungen auf einen Artikel mit Bestand 1 gleichzeitig absetzt: genau eine wird bestätigt, die andere abgelehnt, keine verbleibt in einem Zwischenzustand. Zusätzlich wird dieselbe Bestellung mehrfach abgesendet und darf nur einmal im Bestand erscheinen.

### **NFR-02 (Latenz der Statusaktualisierung)** betrifft FR-16, FR-18

Eine Statusänderung ist beim jeweils anderen Akteur innerhalb von zwei Sekunden sichtbar (95. Perzentil), bei 20 gleichzeitig verbundenen Bar-Geräten.

*Messung:* Lasttest mit simulierten Clients, gemessen wird die Zeit zwischen dem Auslösen der Statusänderung und ihrem Eintreffen beim Empfänger.

### **NFR-03 (Bedienbarkeit unter Festbedingungen)** betrifft FR-10, FR-15

[Ausstehende Recherche]

*Messung:*

### **NFR-04 (Geräte- und Bildschirmunterstützung)**

Alle Funktionen sind auf aktuellem Chrome (Android), Safari (iOS) und einem Desktop-Browser bei Bildschirmbreiten von 320 bis 1920 Pixeln vollständig nutzbar.

*Messung:* Manuelle Testmatrix über die definierten Kombinationen aus Browser, Betriebssystem und Bildschirmbreite.

### **NFR-05 (Nachvollziehbarkeit der Abrechnung)** betrifft FR-21, FR-26, FR-27

Jede Bestandsänderung, also Bezug, Rückgabe und Korrektur, ist dauerhaft mit Zeitpunkt, Bar und auslösendem Benutzer gespeichert, sodass jeder Betrag der Abrechnung auf einzelne Vorgänge zurückführbar ist.

*Messung:* Die Abrechnung einer Testbar wird stichprobenweise auf die zugrunde liegenden Einzelsvorgänge zurückgerechnet.

### **NFR-06 (Betreibbarkeit durch das OK)** betrifft FR-01 bis FR-05, FR-09

Alle wiederkehrenden Betriebsaufgaben, also das Anlegen einer Festausgabe, die Verwaltung von Bars und Sortiment, Bestandskorrekturen und die Abrechnung, sind über die Oberfläche ausführbar und erfordern keinen direkten Datenbankzugriff. Eine nicht am Projekt beteiligte Person richtet anhand der Anleitung eine neue Festausgabe in unter 30 Minuten ein.

*Messung:* Nachweis der Oberflächenabdeckung anhand der Liste der Betriebsaufgaben, Durchführung der Einrichtung durch eine projektfremde Person.

### **NFR-07 (Zugriffsschutz der Bar-Zugänge)** betrifft FR-04

Ein Bar-Zugang ist nicht erratbar (mindestens 122 Bit Entropie) und gewährt ausschliesslich Zugriff auf die Daten der eigenen Bar. Administrative Funktionen sind über diesen Zugang nicht erreichbar.

*Messung:* Negativtests gegen fremde Bar- und Administrationsendpunkte mit einem gültigen Bar-Zugang.

## 1.2.5 Anforderungen (To be removed)

Es wird eine Lösung gesucht, die einfach nutzbar ist (keine versteckten Buttons, suchen von Funktionen..) und während dem Rattenfest muss es fehlerfrei laufen.

Kompromisse

- Wir setzen den Fokus beim Hosting auf Einfachheit und möglichst wenig Setupaufwand für den Server. Dabei nehmen wir in Kauf, uns von einem Hosting Provider mit fertigem Setup abhängig zu machen.
- Bedienbarkeit VS Performance
  - optimieren von Visualisierungen > Getränke Bilder im Cache halten, CDN, Webstandards nutzen (WebP, SVG), Testen wie gross der Einfluss auf die Performance ist oder eine Option zum deaktivieren der Bilder?

weitere Anforderungen (noch priorisieren) :

- Bedienbarkeit:
  - Schritte (sinnvoll) rückgängig machen können
  - Visualisierungen nutzen
  - weniger Klicks und scrolls oder User nicht überfordern? -> Bar User definieren, z.B. wenn es pro Bar eine verantwortliche Person gibt darf die Bedienung schwieriger sein, damit Zeit gespart werden kann
  - auf verschiedenen Geräten und Bildschirmgrößen!!
  - Umstellung auf Digital verbessern
    - Pausenfunktion einbinden für RF Arbeiter, da jetzt Meldungen sofort reinkommen
    - den Bars das einschätzen der Bestellmenge vereinfachen
      - neben der individuellen Auswahl (+1, +6, +24 ...) auch vordefinierte Mengen als Option anbieten. Dazu auch die Überlegung: eine Bestellung von vorher wiederholen zu können.
      - (Bilder nutzen: bei grosser Bestellung ein Bild mit vielen Getränken und bei wenigen Getränken Bild mit wenig Getränken (z.B. mit ein paar vordefinierten Bildern und die einordnen, müsste recherchieren wie das andere lösen))
    - (Bestellung per Touchscreen, andere Bedienfunktionen machen am Rattenfest keinen Sinn. Die Altersgruppe kennt sich mit dem Smartphone aus.)
  - Weitere mögliche Bedienbarkeit Features für Bars
    - Getränke als Favoriten vermerken (automatisch die vorbestellten Getränke markieren) ohne das dieses Feature im Weg ist
    - nach Alkohol und Alkoholfrei filtern (wäre das nützlich?) und sortieren nach Kriterien
    - OK Notfallmeldung per Chat/Telefon/Treffen ermöglichen für Ausnahmefälle und Probleme > einfach halten
      - wenn Website nicht läuft, eine Telefonnummer einblenden die auch offline angezeigt wird.
- wartbarkeit:
  - OK Mitglieder sollten das System einfach updaten können (aber evt. Anleitung erstellen anstatt alles automatisieren) > einfacher geworden mit KI
  - Software best practices anwenden (z.B. Tests)
  - das Meiste soll in einem GUI gelöst werden können, DB Anpassungen nicht per SQL/Table Editor nötig.

- erweiterbarkeit: weitere ineffiziente Prozessabläufe vom Rattenfest digitalisieren/automatisieren ist nicht wichtig, soll möglich sein aber kann auch separat umgesetzt werden. Kommunikation im OK, Website für das Rattenfest, Ticketsystem und weiteres bleibt getrennt.
  - Code Komplexität oder Erweiterbarkeit wichtiger? Zukunftspläne?
- langfristige Lösung: natürlich sollen neue Getränke erfasst werden können, aber auch jährliche Zahlen speichern für Analysen in Zukunft und erweiterbarkeit durch weitere Tools?
- Performance:
  - sollte nicht zu teuer sein, wenn es nur am Rattenfest intensiv genutzt wird, bei ein paar Sekunden längerer Verarbeitungszeit nicht schlimm?
  - das initiale UI und Klicks sollten sofort erkannt werden und sinnvolles Feedback gezeigt werden (lade-animation, Erfolgsmeldung, Überlegung PWA)
    - Internetverbindung während Rattenfest wichtig
  - nicht so langsam, wie ein Webshop sich leisten kann
  - Kurze Teilnehmerumfrage zeigt, dass das Internet während dem Rattenfest an der OST langsamer ist als sonst
- Verbindung
  - Die Nutzer sollten die Verbindung zur Application beibehalten können während dem Rattenfest (Websockets, SSE genügt nicht(?)), so dass sie Status-Nachrichten sofort erhalten ohne regelmäßiges abfragen im Hintergrund. Nachrichten sollen nicht verloren gehen aber eine zu aufwändige Überprüfung (mehrfaches hin-und-zurück zum ob Nachricht ankam) soll nicht die Performance durch erhöhten Traffic verschlechtern (Mittelmaß finden).
    - Trotz frequenten Bestellungen, soll der Lagerbestand immer aktuell sein. Verzögerung bei Bars sind weniger schlimm, als Verzögerungen beim Lagersystem.
    - Datenverkehr: App zu Lager und Bar = Anzahl Getränke vorhanden App zu Bar = Bestellung Ok/Nicht Ok, Bestellung bereit zum abholen App zu Lager = Bestellung erhalten
      1. allfällige Undo/Stornieren Commands
  - Push Nachrichten auf dem Smartphone anzeigen mit PWA wäre ein nützliches Feature.
- Fehler einfach beheben:
  - Hinweise anzeigen, bei ungewöhnlichen Bestellungen (eine 0 zuviel bei Anzahl Getränke, Feld leer gelassen welches bei diesem Getränk sonst eingetragen wird, besonders kleine Bestellung..)
  - Rückgängig/Wiederholen Button genügend gross und schnell (gutes System für History finden)
- Zuverlässigkeit : sehr wichtig, keine halb verarbeiteten Bestellungen, keine falschen Informationen anzeigen
- Redundanz und Skalierbarkeit ist weniger wichtig, da die Anzahl User sich im < 100 Personen Bereich befinden.

## 1.2.6 Functional Requirements

### 1.2.6.1 RF Admin

- Admin Dashboard

Immer:

- Analyse/Statistics: was wäre wichtig? Export als Excel/CSV der Daten (wie? wo speichern?) Vergleiche zwischen verschiedenen Jahren. Bessere Schätzungen

### 1.2.6.2 Bar

- Login mit erhaltenen Daten

- Getränke bestellen (Bar)
- Undo/Redo (siehe oben)
- History > alte Bestellungen betrachten, prüfen, wiederholen
- Abholung > Identität verifizieren nötig? Abholung bestätigen
- Kontaktfunktion (allgemeine Probleme/Fragen melden, Notfallmeldungen während Fest, wer von RF behandelt Meldungen?)

## 1.2.7 Non-Functional Requirements

## 1.3 Anwendungsszenarien (detaillierter)

### 1. Vorbereitung & Inventar

- Ersteinlagerung: Die gesamte eingekaufte Ware wird digital erfasst und bildet das zentrale Live-Inventar.

### 2. Bestellprozess für Bars

- Zugang: Jede Bar erhält einen eigenen Zugangslink (z.B. app.com/UUID).
- Bestelllogik:
  - Bars bestellen flexibel nach Bedarf.
  - Nachbestellungen über die ursprüngliche Schätzung hinaus sind möglich, solange der Artikel im zentralen Inventar verfügbar ist.
  - Optional: Ein automatisches Warnsystem meldet den Organisatoren kritisch tiefe Lagerbestände, erlaubt nur noch eine bestimmte Anzahl pro Bestellung (Falls eigenes Limit bereits erreicht ist).

### 3. Abwicklung & Abholung

1. Bestellung: Die Bar sendet eine Bestellung über die App ab.
  - Bestellung für wenige verschiedene Getränke
  - Bestellung für ein Getränk in grossen Mengen
  - eine Bestellung von vorher anpassen und nochmals bestellen
 a. Bar kann die Bestellung stornieren, anpassen und neu senden.
2. Benachrichtigung: Das RF-Team wird sofort digital informiert.
3. Bereitstellung: Das Team stellt die Ware zusammen und markiert die Bestellung in der App als «Abholbereit». a. RF-Team sieht Problem und kann Bestellung stornieren/markieren.
4. Abholung: Die Bar sieht das Status-Update in Echtzeit und holt die Ware ab, was unnötige Wege vermeidet. a. RF-Team kann Bestellung stornieren/markieren wenn zu lange nicht abgeholt.
0. Bar/RF kann bei technischen Problemen eine Person vom RF mit einer Notfallmeldung erreichen.

### 4. Prozess für Getränkerücknahmen

1. Bars können wie bei einer Bestellung eine Rückgabe erfassen und bringen die Ware zum Lager.
2. Admin-Funktion: Nur ein Admin kann Rücknahmen bearbeiten... (Wichtig bei Rückgabe von Schrumpfpackungen, da diese nur als Einheit angenommen werden)

### 5. Erweitert Analytics, Umsatz pro Bar, pro Getränk etc. Export von Bezugslisten, Bestellhistorie, Rechnung etc. Initiale Bestellung, Getränkewünsche ebenfalls über die App > Analyse, ob ein Getränk viel zu wenig gekauft wurde und durch ein trendigeres Getränk ersetzen vor Schritt 1?

Weitere Features, z.B. «3.1 Bestellung» «die Bestellung für später vorbereiten und noch nicht senden, RF kann diese bereits sehen und sich besser vorbereiten»

- Konkurrenzanalyse durchführen, schauen was die gut machen und wir übernehmen können, ergänzen.

- User Befragung, wäre es sinnvoll oder unnötig
- Abwägen wie gross der Aufwand für den Nutzen ist

## 1.4 Vorhandene Lösungswege

> konkrete gut dokumentierte Lösungen finden

- event organisations softwares > meiste haben nicht fokus Getränkbestellung sondern tickets, zeitmanagement..
- Getränkelieferanten/Webshop Lösungen (Ziel: wenig Performance, es läuft das ganze Jahr durch)
- Lösungen ohne Website?
- der Vorteil gegenüber einem Chat wo man Bestellungen schreibt ist offensichtlich, aber freie Textnachrichten haben auch Vorteile > Bemerkungen-Möglichkeit bei Bestellung einbauen?

### 1.4.1 Webshops

<https://www.rewe.de/shop/c/getraenke-genussmittel/>

- Angenehme Navigation durch farbige Bilder und Icons
  - einzelne Getränke und 6er Packs werden jeweils so auf dem Bild abgebildet

## 2 Analyse

- > Unterschied zur Einleitung?? ist das der Kontext?
- Vielleicht einfach Analyse nennen?

## **2.1 Marktanalyse**

### **2.1.1 Sum-Up**

## **2.2 Recherche User Experience während Rattenfest**

### **2.2.1 Definition**

### **2.2.2 Erkenntnisse**

### **2.2.3 wichtige Probleme**

# **3 Vorarbeiten**

# **4 Methode**

## **4.1 Design und Architektur**

### **4.1.1 Prototyp / Skizze der Lösung**

### **4.1.2 UI und Backend Konzept (Software-Architektur)**

## **4.2 Implementation**

# **5 Ergebnisse**

## **5.1 Evaluation**

### **5.1.1 Functional Requirements**

### **5.1.2 Non-Functional Requirements**

# **6 Fazit und Ausblick**

## **6.1 Retrospektive**

## **6.2 Zukünftige Weiterentwicklung**

# 7 Projektinformationen

## 7.1 Projektplan

Der Aufwand beträgt 2 Arbeitstage pro Woche für ca. 17h/W , insgesamt 240 Stunden Aufwand.

## 7.2 Langfristiger Zeitplan

### 7.2.0.1 M1: Recherche fertig

geschätzte Zeit: xxh benötigte Zeit: xxh

Beschreibung: {Link zu Kapitel in Typst?}

- Recherche erstellt, damit die Anforderungen bestmöglichst priorisiert
- User Tests durchgeführt mit einem Prototyp??
  - neues UI/UX Design oder vorhandenen Prototyp nutzen?
- User Befragung mit Bar-Arbeitern und RF Lager Verwaltern
- Konzept inklusive Application Architektur und Software Architektur anhand der Anforderungen erstellt

### 7.2.0.2 M2: Umsetzung, Programmierung abgeschlossen

Beschreibung:

- FR umgesetzt
- NFR umgesetzt/erreicht
- Tests durchgeführt und Rückmeldungen eingebaut
- Website erreichbar und benutzbar
- allfällige Dokumentation/Anleitungen für User (RF/Bars) fertig

### 7.2.0.3 M3: Dokumentation bereit zum Korrekturlesen

- Erkenntnisse dokumentiert
- Dokumentation gut formatiert
- Abgabe bereit
- Dokumentation kann korrekturgelesen werden
- 1. Puffer

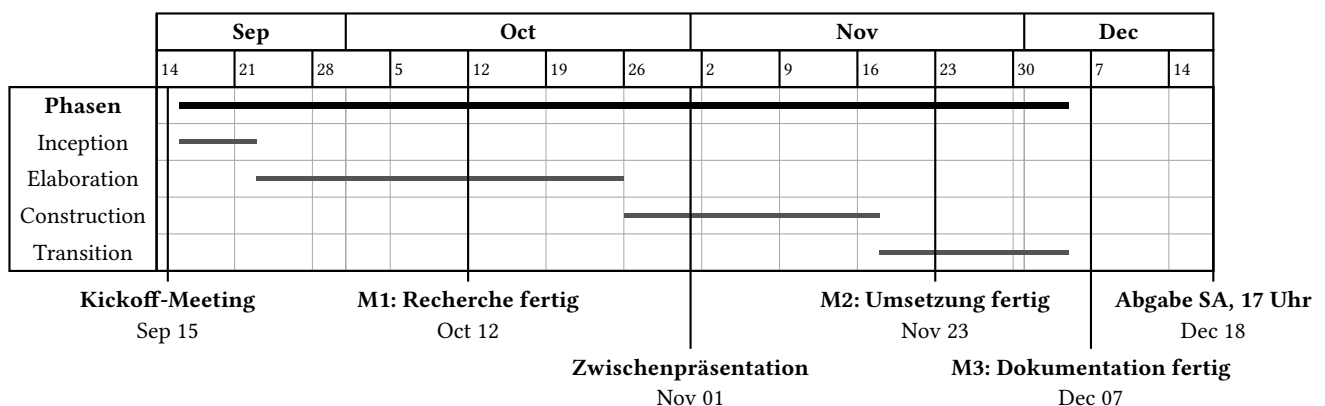


Abbildung 1: Langfristiger Zeitplan

## 7.3 Kurzfristiger Plan

Punkte die erledigt werden pro Woche, Struktur und ausführlichkeit kann sich ändern. Als Hilfe für uns.

Woche 1:

- ☒ Kickoff
- ☒ Dokumentationstruktur aufbauen
- ☒ Aufgabe konkretisieren/Arbeit grob planen
- ☒ Erfolgszenario Entwurf
- ☒ Zeitplan
- ☒ offene Fragen notieren für nächste Besprechung: sind mit > am Anfang geschrieben.

Woche 2:

- ☐ Link zu PDF
- ☐ Dokumentation ergänzen, gemäss Richtlinien und Austausch untereinander
- ☐ interne Kommunikation
- ☐ Ausgangslage und Informationen ausführlich notieren
- ☐ Prototyp vorbereiten für Tests
- ☐ Fragen notieren, was wir mit dem Prototyp herausfinden wollen
- ☐ Fragen notieren, was wir mit der Recherche herausfinden wollen
- ☒ Zeitplan erweitert

Woche 3:

- ☐ User Befragung mit Prototyp
- ☐ Messungen/Tests anhand von Prototyp
- ☐ Ergebnis validieren, sinnvoll notieren
- ☐ Recherche: ausführliche Konkurrenzanalyse

Woche 4:

- ☐ Literaturrecherche aufgrund User Tests
  - Fragen klären
- ☐ Recherche abschliessen (M1)
- ☐ Umsetzung planen

Woche 5: Umsetzung: Details werden in Woche 4 geplant

Woche 6: Umsetzung: Details werden in Woche 4 geplant

- ☐ Zwischenpräsentation vorbereiten

Woche 7: Umsetzung: Details werden in Woche 4 geplant

- ☐ Zwischenpräsentation vorbereiten + halten

Woche 8: Umsetzung: Details werden in Woche 4 geplant

Woche 9: Umsetzung: Details werden in Woche 4 geplant

Woche 10: Umsetzung: Details werden in Woche 4 geplant

- ☐ (M2) Umsetzung fertig

Woche 11:

- ☐ Dokumentation überarbeiten, schönschreiben
- ☐ Dokumentation anpassen gemäss Bewertungskriterien

Woche 12:

- ☐ Dokumentation anpassen gemäss Bewertungskriterien
- ☐ Dokumentation fertig und zum Korrekturlesen geben
- ☐ Dokumentation Formatierung anpassen wenn Zeit
- ☐ (M3) Dokumentation fertig

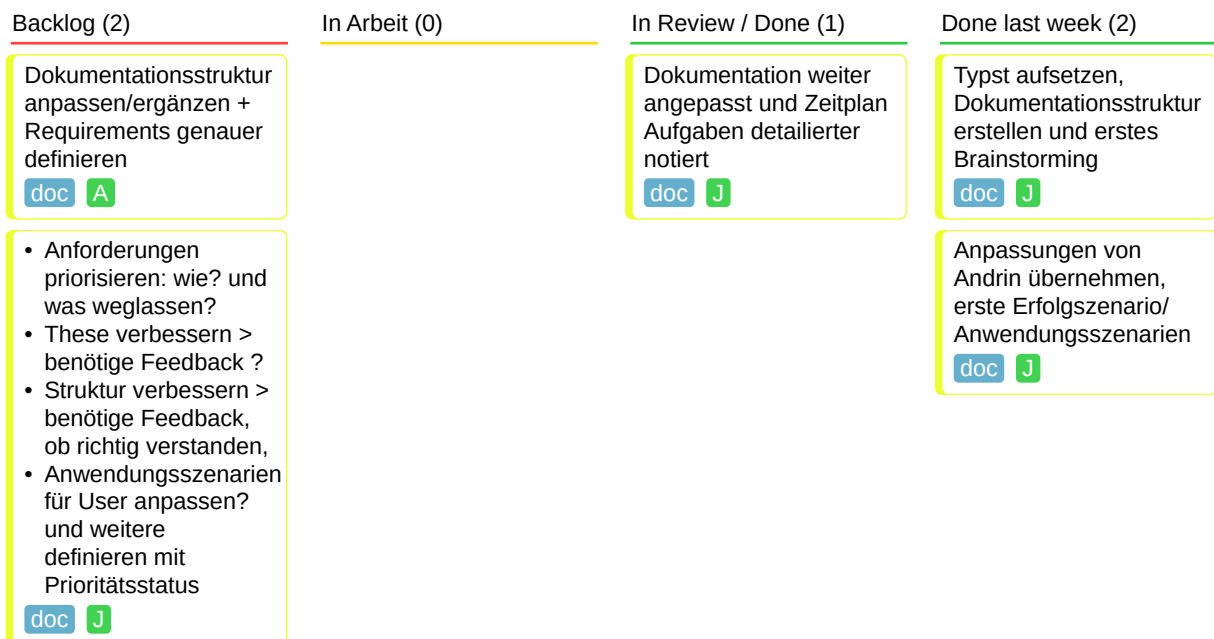
Woche 13:

- ☐ letzte Korrekturen
- ☐ Abgabe vorbereiten (formatieren, exportieren)
  - ☐ Bericht (in einer vollständigen Version zur Korrektur und einer gekürzten Version zur Publikation)
  - ☐ Eigenständigkeitserklärung
  - ☐ A0-Poster (bei Studienarbeiten ist die Erstellung des Posters empfohlen, aber nicht zwingend)
  - ☐ Wissenschaftliches (Plain-Text) Abstract
  - ☐ Broschüren-Abstract

Woche 14 (bis 18. Dezember):

- ☐ Abgabe

### 7.3.1 Kanban Board



## 8 Hilfsmittelverzeichnis

Die folgende Übersicht dokumentiert die im Projekt verwendeten Hilfsmittel nach Aufgabenbereich. Die konkreten Tools werden im Verlauf des Projekts ergänzt.

| Aufgabenbereich                                                     | Tools                               |
|---------------------------------------------------------------------|-------------------------------------|
| Literatur-Recherche und Verwaltung                                  | Google                              |
| Datenanalyse und Visualisierung                                     |                                     |
| Ideengenerierung                                                    | Claude, Gemini                      |
| Übersetzung                                                         |                                     |
| Prototyping                                                         | Figma                               |
| Coding                                                              | Claude Code                         |
| Texterstellung, Textoptimierung, Rechtschreib- und Grammatikprüfung | Claude                              |
| Zusammenarbeit und Projektmanagement                                | Teams, GitHub, Outlook, Google Meet |
| DevOps                                                              | GitHub                              |

Tabelle 3: Hilfsmittelverzeichnis

## 9 Bilderverzeichnis

Abbildung 1 Langfristiger Zeitplan ..... 17

## 10 Tabellenverzeichnis

Tabelle 1 Kategorien der MoSCoW-Klassifikation ..... 9  
Tabelle 2 Klassifikation der Functional Requirements ..... 9  
Tabelle 3 Hilfsmittelverzeichnis ..... 20

# **11 Bibliografie**

## **12 Anhang A**

### **12.1 APIs**

### **12.2 Screenshots**

### **12.3 Testreports**

## **13 Anhang B**

### **13.1 Meeting-Protokolle**

Meetingnotizen werden separat erstellt, hier werden nur wichtige Entscheidungen notiert.

- Wöchentliches Meeting am Dienstag mit Betreuer

### **13.2 Eigenständigkeitserklärung**

### **13.3 Persönliche Bericht**

Zusammenfassung unserer persönlichen Erfahrungen, welche wir aus dem Projekt mitnehmen.

